

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

1. Gestão de Configuração de Software
2. O repositório de SCM
3. O processo SCM
4. DevOps

Gestão de Configuração de Software

Mudanças são inevitáveis quando o software de computador é construído e podem causar confusão quando os membros de uma equipe de software estão trabalhando em um projeto.

A confusão surge quando as mudanças não são analisadas antes de serem feitas, não são registradas antes de serem implementadas, não são relatadas àqueles que precisam saber ou não são controladas de maneira que melhorem a qualidade e reduzam os erros.

Babich¹ discute isso quando afirma:

A gestão de configuração é a arte de identificar, organizar e controlar modificações no software que está em construção por uma equipe de programação. O objetivo é maximizar a produtividade minimizando os erros.

¹Babich, W. A., Software Configuration Management, Addison-Wesley, 1986.

Como as mudanças podem ocorrer a qualquer instante, as atividades de SCM são desenvolvidas para:

1. identificar a alteração
2. controlar a alteração
3. assegurar que a alteração esteja sendo implementada corretamente e
4. relatar as alterações a outros envolvidos.

É importante fazer uma distinção clara entre suporte de software e gestão de configuração de software.

Suporte é um conjunto de atividades de engenharia que ocorrem depois que o software é fornecido ao cliente e posto em operação.

Gestão de configuração é um conjunto de atividades de rastreamento e controle. Essas atividades se iniciam quando um projeto de engenharia de software começa e terminam apenas quando o software sai de operação.

Um dos principais objetivos da engenharia de software é **incrementar a facilidade com que as alterações podem ser acomodadas e reduzir o esforço necessário quando alterações tiverem de ser feitas.**

Gestão de configuração de software

O processo de software resulta em informações que podem ser divididas em três categorias principais:

- Programas de computador (tanto na forma de código-fonte quanto na forma executável)
- Produtos que descrevem os programas de computador (focado em vários envolvidos)
- Dados ou conteúdo (contidos nos programas ou externos a ele)

Os **itens que compõem** todas as informações produzidas como parte do processo de software são chamados coletivamente de **configuração de software**.

À medida que o trabalho de engenharia de software avança, cria-se uma hierarquia de **itens de configuração de software** (SCIs, *software configuration items*).

Um elemento de informação pode ser tão pequeno quanto um simples diagrama UML ou tão grande quanto um documento de projeto completo.

Se cada SCI simplesmente conduzir a outros SCIs, resultará em pouca confusão.

Infelizmente, outra variável entra no processo – **alteração**.

A alteração pode ocorrer a qualquer momento e por qualquer razão.

De fato, a Primeira Lei da Engenharia de Sistemas² diz:

Não importa onde você esteja no ciclo de vida do sistema, o sistema mudará, e o desejo de alterá-lo persistirá por todo o ciclo de vida.

²Bersoff, E., V. Henderson, and S. Siegel, Software Configuration Management, Prentice Hall, 1980.

A gestão de configuração de software é um conjunto de **atividades** desenvolvidas para **gerenciar alterações** ao longo de todo o **ciclo de vida** de um software.

A SCM pode ser vista como uma **atividade de garantia de qualidade do software** aplicada em todo o processo do software.

A mudança ou alteração é um fato normal no desenvolvimento de software.

- Clientes querem modificar requisitos.
- Desenvolvedores querem modificar a abordagem técnica.
- Gerentes querem modificar a abordagem do projeto.

Por que todas essas modificações?

À medida que o tempo passa, todas as partes envolvidas sabem mais:

- O que precisam
- Qual será a melhor abordagem
- Como ganhar dinheiro com o software

Esse **conhecimento adicional é a força motriz que está por trás da maioria das alterações** e leva à constatação de um fato que para muitos profissionais de engenharia de software é difícil de aceitar: **muitas alterações são justificadas!**

Uma **referência** é um conceito de gestão de configuração de software que ajuda a controlar alterações sem obstruir seriamente as alterações justificáveis.

O IEEE (IEEE Std. No. 610.12-1990) define uma referência como:

Uma especificação ou produto que tenha sido formalmente revisado e acordado, que depois serve de base para mais desenvolvimento e pode ser alterado somente por meio de procedimentos formais de controle de alteração.

Para que um item de configuração de software se torne uma referência para o desenvolvimento, as alterações devem ser feitas rápida e informalmente.

No entanto, uma vez estabelecida uma referência, podem ser feitas alterações, mas **deve ser aplicado um processo específico e formal para avaliar e verificar cada alteração.**

No contexto da engenharia de software, uma **referência** é um **marco significativo no desenvolvimento de software**.

Uma referência é marcada pelo **fornecimento de um ou mais itens de configuração de software** que foram aprovados em consequência de uma revisão técnica.

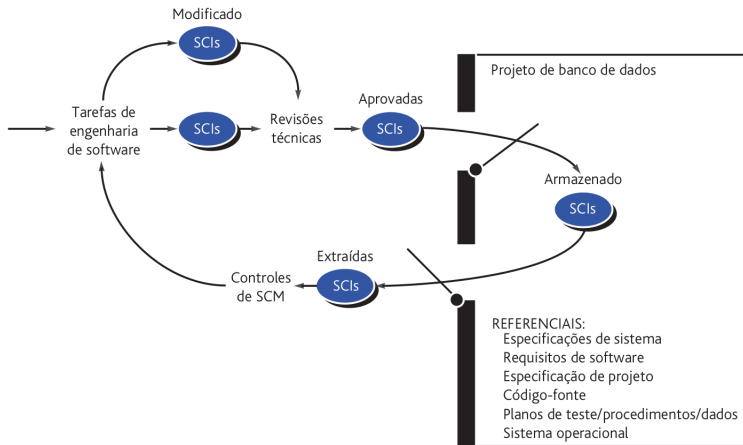


Figura 1: SCIs que se tornaram referenciais e o banco de dados de projeto.

Em uma visão mais realista, um SCI é todo ou parte de um artefato (por exemplo, um documento, um conjunto inteiro de casos de teste ou um programa ou componente com nome).

Itens de configuração de software

Além dos SCIs derivados dos artefatos de software, **ferramentas de software** também podem estar sob o controle de configuração.

Isto é, versões específicas de editores, compiladores, navegadores e outras ferramentas automáticas são “congeladas” como parte da configuração do software.

Como essas ferramentas foram usadas para produzir documentação, código-fonte e dados, devem estar disponíveis quando alterações forem feitas na configuração do software.

Gestão de dependências e alterações

A **matriz de rastreabilidade** é um modo de documentar dependências entre requisitos, decisões de arquitetura e causas de defeito.

Essas dependências precisam ser levadas em conta ao se determinar o impacto de uma alteração proposta e orientar a escolha dos casos de teste que devem ser usados para teste de regressão.

Rastreabilidade é um termo da engenharia de software que se refere a **mapeamentos** documentados **entre os artefatos** de engenharia de software (por exemplo, requisitos e casos de teste).

Uma matriz de rastreabilidade permite a um engenheiro de requisitos representar a relação entre os requisitos e outros artefatos da engenharia de software.

O repositório de SCM

O repositório de SCM

O repositório de SCM é um conjunto de mecanismos e estruturas de dados que permitem a uma equipe de software **gerenciar alterações** de maneira eficaz.

Ele fornece as funções óbvias de um sistema moderno de gestão de banco de dados, garantindo a integridade dos dados, compartilhamento e integração.

Além disso, o repositório de SCM proporciona um centralizador (hub) para a integração das ferramentas de software, está no centro do fluxo do processo de software e pode impor estrutura e formato uniformes para os artefatos.

As características e o conteúdo do repositório são mais bem compreendidos quando observados a partir de duas perspectivas: o que tem de ser armazenado no repositório e quais são os serviços específicos fornecidos pelo repositório.

A Figura 2 mostra uma divisão detalhada dos tipos de representações, documentos e outros produtos que são armazenados no repositório.

Características gerais e conteúdo

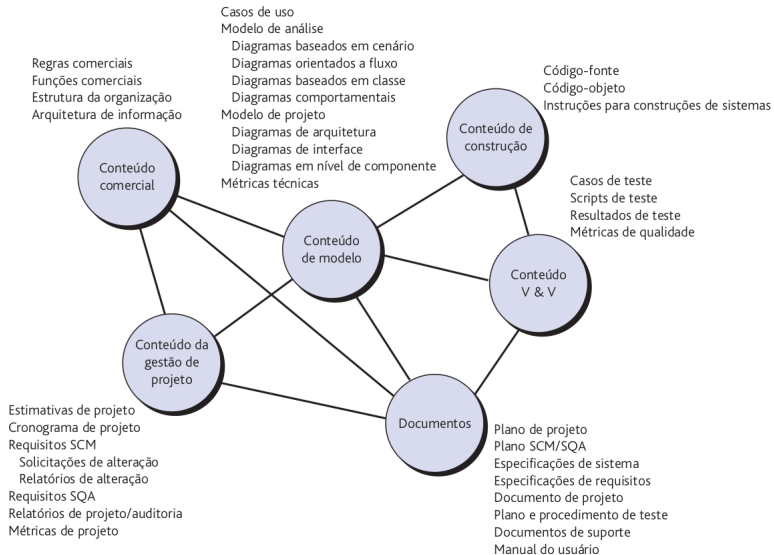


Figura 2: Conteúdo do repositório.

Para suportar a SCM, o repositório deve ter um conjunto de ferramentas que dê suporte para estas características:

Versões. À medida que um projeto avançar, serão criadas muitas versões dos artefatos individuais. O repositório deve ser capaz de salvar todas essas versões para possibilitar uma gestão eficaz das versões do produto e permitir aos desenvolvedores retroceder a versões anteriores durante o teste e depuração.

O repositório deve poder controlar uma grande variedade de tipos de objetos, incluindo texto, gráficos, bitmaps, documentos complexos e objetos especiais, como definições de tela e relatório, arquivos de objeto, dados de testes e resultados.

Acompanhamento de dependências e gestão de alterações. O repositório gerencia uma grande variedade de relações entre os elementos de dados nele armazenados. Isso inclui relações entre entidades e processos corporativos, entre as partes do projeto de uma aplicação, entre componentes de projeto e arquitetura de informações corporativas, entre elementos de projeto e outros artefatos e assim por diante.

Algumas dessas relações são meramente associações, e outras são relações de dependência ou de obrigatoriedade.

Controle de requisitos. Essa função especial depende da gestão de links e proporciona a capacidade de controlar todos os componentes de projeto e construção e outros produtos que resultam de uma especificação especial de requisitos (acompanhamento adiante). Além disso, proporciona a capacidade de identificar quais requisitos geraram determinado produto (acompanhamento retroativo).

Gestão de configuração. O recurso de gestão de configuração mantém controle de uma série de configurações representando marcos de projeto específicos ou versões de produção.

Pistas de auditoria. Uma pista de auditoria estabelece informações adicionais sobre quando, por que e por quem foram feitas as alterações. As informações sobre a origem das alterações podem ser colocadas como atributos de objetos específicos no repositório. Um mecanismo de disparo de repositório é útil para avisar o desenvolvedor ou a ferramenta que está sendo usada para iniciar a aquisição de informações de auditoria (como, por exemplo, a razão de uma alteração) sempre que um elemento de projeto for modificado.

O processo SCM

O processo de gestão de configuração de software define uma série de tarefas que têm quatro objetivos principais:

1. identificar todos os itens que coletivamente definem a configuração do software,
2. gerenciar alterações de um ou mais desses itens,
3. facilitar a construção de diferentes versões de uma aplicação e
4. assegurar que a qualidade do software seja mantida à medida que a configuração evolui com o tempo.

O processo SCM

Um processo que atinja esses objetivos não precisa ser burocrático e pesado, mas deve ser caracterizado de maneira que permita à equipe de software desenvolver respostas a várias questões complexas:

- Como uma equipe de software identifica os elementos discretos de uma configuração de software?
- Como uma organização lida com as várias versões de um programa (e sua documentação) de maneira que venha a permitir que a alteração seja acomodada eficientemente?
- Como uma organização controla alterações antes e depois que o software é entregue ao cliente?
- Como uma organização avalia o impacto das alterações e gerencia o impacto efetivamente?

- Quem tem a responsabilidade de aprovar e classificar as alterações solicitadas?
- Como podemos assegurar que as alterações foram feitas corretamente?
- Que mecanismo é usado para alertar outras pessoas sobre as alterações feitas?

A partir dessas questões são definidas cinco tarefas SCM – identificação, controle de versão, controle de alteração, auditoria de configuração e relatos.

O processo SCM

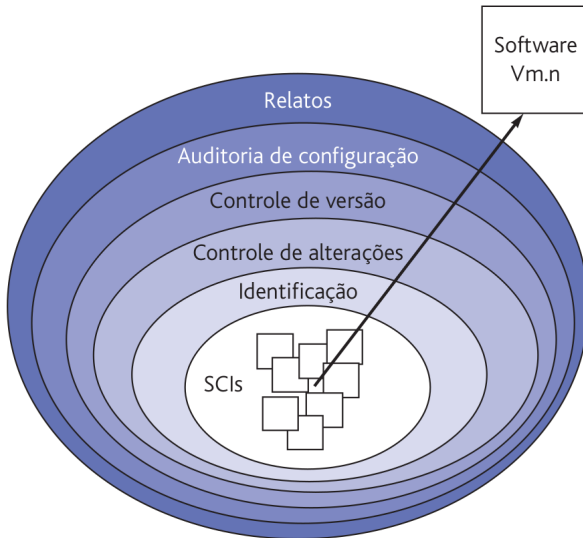


Figura 3: Camadas do processo SCM.

Uma vez que o objeto tenha passado pela revisão técnica e tenha sido aprovado, pode ser criado um referencial. Uma vez que um SCI se torna um referencial, é implementado o controle de alterações em nível de projeto.

Em alguns casos, o desenvolvedor prescinde da geração formal de pedidos de alteração e relatórios de alterações. No entanto, é feita a avaliação de cada alteração e todas as alterações são acompanhadas e revisadas.

Quando o artefato de software é liberado para os clientes, institui-se o controle formal de alterações.

Controle de versão

O controle de versão combina procedimentos e ferramentas para gerenciar diferentes versões dos objetos de configuração criados durante o processo de software.

Um sistema de controle de versão implementa ou está diretamente integrado a quatro recursos principais:

1. um banco de dados de projeto (repositório) que armazena todos os objetos de configuração relevantes,
2. um recurso de gestão de versão que armazena todas as versões de um objeto de configuração (ou permite que qualquer versão seja construída usando diferenças das versões anteriores),
3. uma facilidade de construir que permite coletar todos os objetos de configuração relevantes e construir uma versão específica do software.

Identificação, controle de versão e controle de alterações ajudam a manter a ordem naquilo que, de outra forma, seria uma situação caótica.

Mas como a equipe de software pode assegurar que a alteração foi implementada corretamente? A resposta é dupla:

1. revisões técnicas e
2. a auditoria de configuração de software.

A revisão técnica focaliza a exatidão técnica do objeto de configuração modificado.

Os revisores avaliam o SCI para determinar a consistência com outros SCIs, omissões ou efeitos colaterais em potencial.

Deve ser feita uma revisão técnica para todas as alterações, exceto as mais triviais.

O relatório de status de configuração (CSR) é uma tarefa da SCM que responde às seguintes questões:

1. O que aconteceu?
2. Quem fez?
3. Quando aconteceu?
4. O que mais será afetado?

A saída do CSR pode ser colocada em um banco de dados online ou em um site, de forma que os desenvolvedores de software ou pessoal de suporte possam acessar as informações de alterações por categoria de palavra-chave.

Além disso, é gerado um relatório do CSR regularmente; ele se destina a manter a gerência e os profissionais informados sobre alterações importantes.

DevOps

Imagine a world where product owners, development, QA, IT Operations, and Infosec work together, not only to help each other, but also to ensure that the overall organization succeeds.

Gene Kim, Jez Humble, Patrick Debois, John Willis

Apesar de ser um termo razoavelmente novo, existe uma tendência em ver **DevOps** como um movimento que visa introduzir práticas ágeis na última etapa de um projeto de software, isto é, quando o sistema vai entrar em produção.

Após um *sprint*, o sistema ou um incremento dele está pronto para entrar em produção.

Essa tarefa é conhecida como implantação (**deploy**), liberação (**release**) ou entrega (**delivery**) do sistema.

Independentemente do nome, ela não é tão simples e rápida como pode parecer.

Em organizações tradicionais, a área de Tecnologia da Informação era dividida em dois departamentos:

- Departamento de Sistemas (ou Desenvolvimento): formado por desenvolvedores, programadores, analistas, arquitetos, etc.
- Departamento de Suporte (ou Operações): formado por administradores de rede, administradores de bancos de dados, técnicos de suporte, técnicos de infraestrutura, etc.

Na maioria das vezes, a área de suporte tomava conhecimento de um sistema na véspera da sua implantação.

Consequentemente, a implantação poderia atrasar por meses, devido a uma variedade de problemas que não tinham sido identificados, como problemas de desempenho, incompatibilidades, etc.

No limite, esses problemas poderiam resultar no cancelamento da implantação e no abandono do sistema.

Para facilitar a implantação e entrega de sistemas, foi proposto o conceito de **DevOps**.

Seus proponentes³ gostam de descrever DevOps como um movimento que visa unificar as culturas de desenvolvimento (Dev) e de operação (Ops), visando permitir a implantação mais rápida e ágil de um sistema.

Segundo eles:

Em vez de iniciar as implantações à meia-noite de sexta-feira e passar o fim de semana trabalhando para concluí-las, as implantações ocorrem em qualquer dia útil, quando todos estão na empresa e sem que os clientes percebam — exceto quando encontram novas funcionalidades e correções de bugs.

³KIM, Gene et al. The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations, 2016.

DevOps não advoga a criação de um profissional novo, que fique responsável tanto pelo desenvolvimento como pela implantação de sistemas.

Em vez disso, defende-se uma aproximação entre o pessoal de desenvolvimento e o pessoal de operações e vice-versa, visando fazer com que a implantação de sistemas seja mais ágil e menos traumática.

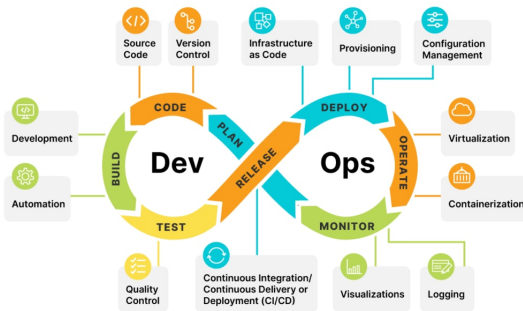


Figura 4: Ciclo de vida DevOps.

Antes de DevOps, Humble e Farley descreveram um conjunto de princípios sobre entrega contínua de software:

- Crie um processo repetível e confiável para entrega de software: não pode ser traumático.
- Automatize tudo que for possível.
- Mantenha tudo em um sistema de controle de versões.
- Se um passo causa dor, execute-o com mais frequência o quanto antes.
- “Concluído” significa pronto para entrega.
- Todos são responsáveis pela entrega.

Controle de versões

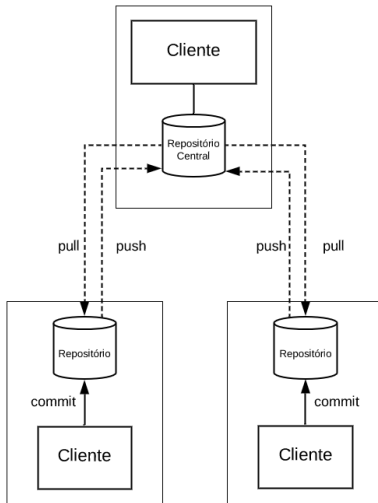


Figura 5: Controle de versões.

Antes de implementar uma nova funcionalidade, pode ser comum criar um *branch* para conter o seu código, denominados de *feature branches*.

Dependendo da complexidade da *feature*, eles podem levar meses para serem integrados à *main*.

Logo, em sistemas maiores e mais complexos podem existir dezenas de *branches* ativos.

Integração Contínua

Na integração, uma variedade de conflitos pode ocorrer, os quais são conhecidos como **conflitos de integração** ou **conflitos de merge**.

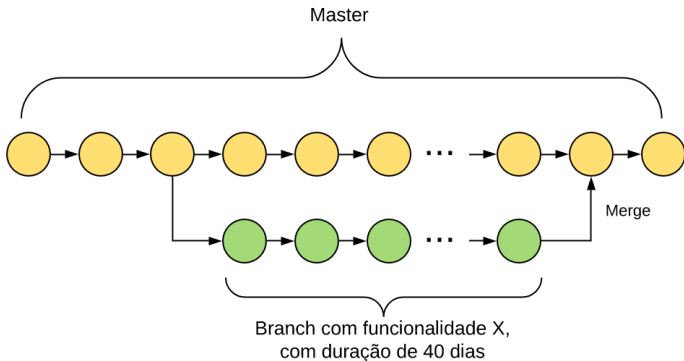


Figura 6: Conflito de integração de uma nova funcionalidade.

Integração Contínua (CI) é uma prática de desenvolvimento proposta por Extreme Programming (XP).

O objetivo é não deixar que a tarefa que causa dor se acumule.

Em vez disso, devemos quebrá-la em subtarefas que possam ser realizadas de forma frequente.

Como essas subtarefas são pequenas e simples, a dor decorrente da sua realização será menor.

De acordo com Kent Beck no seu livro de XP⁴:

Programação em times não é um problema do tipo dividir-e-conquistar. Na verdade, é um problema que requer dividir, conquistar e integrar. A duração de uma tarefa de integração é imprevisível e pode facilmente levar mais tempo do que a tarefa original de codificação. Assim, quanto mais tempo você demorar para integrar, maiores e mais imprevisíveis serão os custos.

⁴BECK, Kent. Extreme programming explained: embrace change. Addison-wesley, 2000.

Em integração contínua, recomenda-se que todo o processo seja automatizado:

- Build Automatizado: *build* é o nome usado para designar a compilação de todos os arquivos de um sistema, até a geração de uma versão executável.
- Testes automatizados: além de compilar, é importante garantir que o sistema continua com o comportamento esperado.

Com integração contínua, código novo é frequentemente integrado no branch principal.

Porém, esse código não precisa estar pronto para entrar em produção.

Ele pode ser uma versão preliminar, que foi integrado para que os outros desenvolvedores tomem ciência da sua existência e, conseqüentemente, evitem conflitos de integração futuros.

Por isso, na cadeia de automação proposta por DevOps existe o *Deployment* Contínuo (CD).

Quando usa-se CD, todo novo *commit* que chega na main entra rapidamente em produção.

O fluxo de trabalho quando se usa CD é o seguinte:

- O desenvolvedor desenvolve e testa na sua máquina local.
- Ele realiza um commit e o servidor de CI executa novamente um build e os testes de unidade.
- Algumas vezes no dia, o servidor de CI realiza testes mais exaustivos com os novos commits que ainda não entraram em produção: testes de integração, testes de interface e testes de desempenho.
- Se todos os testes passarem, os commits entram imediatamente em produção. E os usuários já vão interagir com a nova versão do código.

Algumas ferramentas de automação, integração e *deploy*:

- Ansible (automação e gerenciamento de configuração)⁵
- Jenkins (automação, CI/CD)⁶⁷
- GitHub Actions (CI/CD do Github) ⁸

⁵<https://www.ansible.com/>

⁶<https://www.jenkins.io/>

⁷<https://youtu.be/6YZvp2GwT0A?si=gGL5lmPLLtnqztce>

⁸<https://docs.github.com/en/actions>

Referências

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides.
Padrões de Projetos: Soluções Reutilizáveis de Software Orientados a Objetos, volume 1.
Bookman, 1. edition, 2000.
- [2] R. S. Pressman and B. R. Maxim.
Engenharia de Software uma abordagem profissional, volume 1.
AMGH Editora Ltda, 9. edition, 2021.
- [3] S. R. Schach.
Engenharia de Software: Os paradigmas clássicos & orientados a objetos, volume 1.
McGraw-Hill, 7. edition, 2008.
- [4] I. Sommerville.
Engenharia de Software, volume 1.
São Paulo: Addison-Wesley, 10. edition, 2019.

- [5] M. T. Valente.
Engenharia de software moderna, volume 1.
Independente, 2020.
- [6] R. S. Wazlawick.
Engenharia de Software - Conceitos e Prática, volume 1.
Rio de Janeiro: GEN LTC, 2. edition, 2019.

Perguntas?

Obrigado pela atenção!